HW3 – Group 5

Calabrese Lorenzo (s345902) **Calfapietro Flavia** (s345862) **Spedicato Giuseppe** (s337116)

1. System Architecture

The project implements an end-to-end IoT system for environmental monitoring. The architecture is structured into three main layers:

1. **Edge Layer**: A Raspberry Pi that samples data from the DHT-11 sensor via the `publisher.py` script, publishing to the MQTT topic `s337116` every 5 seconds.
2. **Ingestion & Storage**: An `MQTT Subscriber` that receives JSON messages and populates a Redis database. It uses the `redis.ts()` module to create time series specific to the MAC address (e.g., `0xe45f01d8f381:temperature`).
3. **Service Layer**: A REST server (`cherrypy`) that exposes endpoints for sensor management (`/sensors`, `/sensor`) and historical data retrieval (`/data`).

2. Data Communication & Management

Edge-to-cloud communication is asynchronous. The payload sent by the sensor includes a timestamp in milliseconds, the MAC address and a list of `name/value` dictionaries. The REST server has been configured with a `MethodDispatcher`, using a tree structure with 4 main endpoints to correctly manage CRUD operations on time series, sensors and system status; this allows the end client to perform an appropriate visualization of the data received from the server.

3. Discussion of the Questions

3.1. MQTT vs REST for Data Transmission

The **MQTT** protocol was preferred over REST for transmitting data from the Raspberry Pi to the cloud for the following reasons:

- **Reduced Overhead**: MQTT has a binary header of only 2 bytes, making it much more efficient than HTTP (REST), which requires bulky textual headers. This optimizes bandwidth usage during frequent sampling (every 5s). In addition, the absence of periodic client polling reduces unnecessary message exchanges, further lowering communication overhead in scenarios with frequent updates.
- **Publish/Subscribe Model**: It decouples devices. The RPI does not need to know the server's IP address or wait for a synchronous response; it simply publishes data to the broker (`broker.emqx.io`). This asynchronous interaction enables faster data propagation, allowing sensor readings to be delivered with lower latency than the traditional request-response pattern.
- **Reliability on Unstable Networks**: MQTT better handles connection losses typical of IoT nodes through persistent sessions and keep-alive mechanisms, consuming less energy compared to the continuous opening and closing of TCP sockets typical of REST. Furthermore, MQTT is designed to operate with limited computational and memory resources, making it particularly suitable for constrained devices such as the Raspberry Pi and the DHT-11 sensor.

3.2. Choice of HTTP Method for `/data/{mac}`

For the historical data retrieval endpoint, the **GET** method was implemented.

- **Rationale**: According to REST standards, the GET method is intended for the **retrieval** of resources. It is a "safe" and "idempotent" operation, meaning that repeated requests with the same parameters do not alter the server state and always produce consistent results. Other HTTP methods are not suitable in this context, as POST is typically used to create new resources, while PUT and DELETE are meant for modifying or removing existing ones.
- **Implementation**: The endpoint receives the MAC address in the path and the `count` parameter in the query string. The server executes a `revrange()` command on Redis to extract the data without altering the stored samples, fully respecting the semantics of the GET method.

4. Evaluation and Results

The system was successfully validated. A REST client was implemented to validate the correctness of the exposed API and to visualize the collected data. The `rest_client.ipynb` confirmed that: 1. The server is online; 2. The sensor is correctly registered (HTTP 200 or 409 if already existing); 3. The data are correctly retrieved and visualized using `matplotlib`.

Parameter	Test Value
MAC Address	0xe45f01d8f381
MQTT Broker	broker.emqx.io
Sampling Rate	5.0 seconds

Table 1: System configuration parameters.